

Báo cáo rà soát test code

Lỗi nghiêm trọng

- `test_missing_subtotal`: Kiểm tra khi trường `*subtotal*` trống và kiểm tra không có lỗi (`error_text == ""`). Theo yêu cầu, vì cần thiêu dữ liệu số bắt buộc (như `*subtotal*`) phải hiển thị thông báo lỗi, vì vậy test này sai logic và sẽ luôn “pass” sai lệch so với yêu cầu.
- Thiêu kiểm tra TC-005 (Submit form with missing required numeric input – subtotal undefined): Test plan có mô tả rõ ràng kiểm tra này, nhưng trong mã kiểm tra không có bất kỳ test nào kiểm tra thông báo lỗi khi `*subtotal*` không được cung cấp. Đây là một trường hợp negative quan trọng bị bỏ sót.
- `test_negative_weight_exception`: Sẽ dùng `JavaScriptException` để bắt lỗi JavaScript khi gọi `shippingFee(0, ...)`. Selenium không thể ném `JavaScriptException` cho lỗi JavaScript trong `execute_script`; thay vào đó hàm sẽ trả về giá trị null và lỗi sẽ được ghi vào console. Vì vậy test này sẽ luôn thất bại hoặc không kiểm tra đúng hành vi. Cần thay cách kiểm tra (ví dụ: thực thi hàm qua `driver.execute_script` và kiểm tra `console.error` hoặc tạo một test UI gây lỗi và kiểm tra error output).

Test còn thiêu

- Kiểm tra TC-005: Khi `*subtotal*` không được nhập, phải xác nhận rằng phần `#error` hiển thị thông báo thích hợp và `#total` không thay đổi.
- Kiểm tra định dạng hiển thị: Xác nhận rằng giá trị trong `#total` luôn có đúng hai chữ số thập phân (ví dụ: “115.00” chứ không phải “115”).
- Kiểm tra trường hợp trọng lượng = 0 (đúng như rule `weight must be greater than zero`) thông qua UI, không chỉ qua `execute_script`.
- Kiểm tra trường hợp `credit > subtotal` dẫn tới tổng âm và được làm tròn thành 0.00 (đã có TC-006 nhưng nên xác nhận chuỗi hiển thị “0.00” chứ không chỉ so sánh float).
- Kiểm tra đầu vào không hợp lệ cho destination hoặc tier (ví dụ giá trị không tồn tại) để chắc chắn không gây lỗi JavaScript không bắt được.

Assertion yếu

- Các assert so sánh `float(total_text) == expected` không kiểm tra định dạng chuỗi (hai chữ số thập phân). Nếu UI trả về “115” thay vì “115.00” test vẫn sẽ pass vì float conversion bỏ qua định dạng.
- `assert error_text == ""` trong `test_missing_subtotal` và các test khác không kiểm tra nội dung chi tiết của thông báo lỗi (nếu có). Nên so sánh với thông báo dự kiến.

Rủi ro maintainability

- Hardcoded HTML trong fixture: Mã HTML được nhúng trực tiếp trong fixture; nếu giao diện thay đổi (thêm class, thay đổi id) tất cả test sẽ phá vỡ. Nên để file HTML thực tế hoặc sử dụng URL tĩnh.
- Sử dụng Edge WebDriver: Nếu môi trường CI không có Edge driver, test sẽ không chạy. Nên chuyển sang Chrome/Firefox hoặc cấu hình driver qua bên môi trường.
- Không tách logic tính toán: Các hàm `calculate_discount` và `shipping_fee` được sao chép trong test; nếu logic trong source thay đổi, các hàm này sẽ không theo kịp, gây lệch kết quả kiểm tra.

Gợi ý sửa ngay

1. Sửa `test_missing_subtotal`

```
def test_missing_subtotal(driver):
    data = {
        "subtotal": "", # trường
        "weight": 10,
        "destination": "domestic",
        "tier": "standard",
        "fragile": False,
```

```

"credit": 0,
}
fill_form(driver, data)
driver.find_element(By.CSS_SELECTOR, "#calculate").click()
total_text = driver.find_element(By.CSS_SELECTOR, "#total").text
error_text = driver.find_element(By.CSS_SELECTOR, "#error").text

```

```

assert error_text != "", "Expected an error message when subtotal is missing"
# total không thay đổi, nên so sánh với giá trị mặc định ban đầu (0.00)
assert total_text == "0.00"

```

2. Thêm test cho TC-005 (kiểm tra thông báo lỗi khi subtotal không có):

```

@pytest.mark.parametrize("case_id,input_data,expected_error", [
("TC-005", {"subtotal": "", "weight": 10, "destination": "domestic",
"tier": "standard", "fragile": False, "credit": 0},
"Subtotal is required") # thông báo mẫu, thay đổi theo thực tế
])

```

```

def test_validation_errors(driver, case_id, input_data, expected_error):
fill_form(driver, input_data)
driver.find_element(By.CSS_SELECTOR, "#calculate").click()
error_text = driver.find_element(By.CSS_SELECTOR, "#error").text
assert expected_error in error_text

```

3. Kiểm tra định dạng hai chữ số thập phân:

```

assert re.fullmatch(r"\d+\.\d{2}", total_text), "Total must have exactly two decimal places"

```

4. Thay đổi test_negative_weight_exception để kiểm tra UI thay vì JavaScriptException:

```

def test_negative_weight_shows_error(driver):

```

```

data = {
"subtotal": 100,
"weight": 0, # không hợp lệ
"destination": "domestic",
"tier": "standard",
"fragile": False,
"credit": 0,
}

```

```

fill_form(driver, data)
driver.find_element(By.CSS_SELECTOR, "#calculate").click()
error_text = driver.find_element(By.CSS_SELECTOR, "#error").text
assert "weight must be greater than zero" in error_text

```

5. Tách HTML ra file riêng và tải bằng driver.get("file:///path/to/ui_checkout_fixture.html") để giảm phụ thuộc vào chuỗi HTML dài trong code.

Áp dụng các sửa đổi trên sẽ loại bỏ các lỗi nghiêm trọng, loại bỏ các khách báo còn thiếu, cải thiện mức minh bạch assertion và giảm rủi ro khi báo lỗi.